ELI — Coding Agent

ELI v2.5.11

October 2026

Contents

ELI Coding Agent (`eli/coding/`) **1**

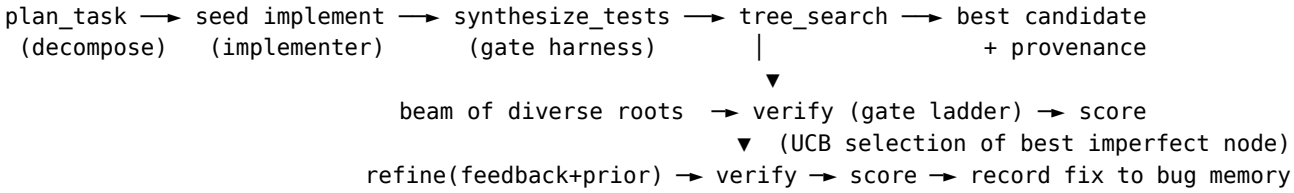
The loop . . . 1
Components . . . 1
Capability checklist (as requested) . . . 3
Wiring & control . . . 3
Honest scope (what's verified vs model-dependent) . . . 3
FIX_FILE (`executor_enhanced.py`, not this package, but shares its safety net) . . . 3

ELI Coding Agent (`eli/coding/`)

> **Updated for v2.4.84.** Invoked via `CODE_SOLVE` / `GENERATE_SCRIPT`; separate from the 15-agent chat bus (its own pipeline: plan → search → verify → repair).

A self-contained, **additive** subsystem that lifts ELI's code generation, analysis, and repair to frontier-grade. It touches nothing that already works: it lives in its own package and is reached through a new `CODE_SOLVE` action. The LLM is an *injected* `generate` callable (default = the local inference broker), so the whole pipeline is model-agnostic and unit-testable without a model.

The loop



Components

File	Capability
`sandbox.py`	**Mandatory execution-feedback substrate.** Runs code in a bounded, isolated subprocess (temp cwd, scrubbed env, `MPLBACKEND=Agg`, wall-clock timeout, POSIX `RLIMIT_CPU`; **no `RLIMIT_AS`** — it breaks numpy). Multi-language via a runner map (Python deepest). Crash detection is precise: only a genuine Python traceback counts; timeouts / signal kills / missing optional deps are tolerated so legitimate long/heavy/optional code isn't falsely failed. `smoke_import` verifies a patched module still loads.
`bug_memory.py`	**Semantic bug classification + long-term memory.** `classify_bug` maps a failure to a `BugClass` (null-handling, type, index/off-by-one, key-missing, name-undefined, import, zero-division, value, assertion, recursion, infinite-loop, logic-inversion, state-corruption, concurrency, resource-leak, io) with a stable, fuzzy-matchable *signature* (exception type + last frame + normalized message). `BugMemory` is a local SQLite store of (signature → class → fix) with exact-then-fuzzy (difflib) recall, so the agent learns from past fixes.
`verification.py`	**Explicit verification gating + scoring + test synthesis.** `verify_candidate` runs the ladder syntax → execution → tests, stopping at the first failure and attaching a `BugDiagnosis` + repair feedback. `score_candidate` aggregates a 0..1 score (tests dominate when present, else clean execution). `synthesize_tests` produces an executable harness (LLM-driven; deterministic smoke-harness fallback) that prints `ELI_TESTS: <passed>/<total>`.
`planner.py`	**Structured decomposition — planner/implementer separation.** `plan_task` → an approach + ordered steps (JSON, with a single-step fallback). `implement` writes/refines code against the plan; in refine mode it does **patch-based incremental refinement** (fix the specific feedback, keep prior code).
`search.py`	**Tree search over solutions (not single-shot).** A diverse beam of roots (temperature ladder = exploration), then iterative expansion choosing the node to refine by **UCB1** (`score + c·√(ln N / nᵢ)`) — exploitation vs exploration. Refinements pull matching prior fixes from bug memory into the feedback; successful repairs are written back as (bug → unified-diff fix) pairs. Stops on `target_score` or budget.
`agent.py`	**`CodeAgent` / `solve()`** — composes all of the above; default broker-backed generator; returns best code + full provenance (plan, search trace, score, bug class).

Capability checklist (as requested)

- structured decomposition (planner/implementer separation) — `planner.py` [done]
- execution feedback loop (mandatory) — `sandbox.py` + `verify_candidate` gate 2 [done]
- tree search over solutions (not single shot) — `search.py` beam + UCB [done]
- explicit verification gating — `verification.py` gate ladder [done]
- long-term memory of bugs and fixes — `bug_memory.py` [done]
- automated test synthesis — `verification.synthesize_tests` [done]
- patch-based incremental refinement — `planner.implement` refine mode + search expansion [done]
- semantic bug classification (null handling, logic inversion, state corruption, …) — `bug_memory.classify_bug` [done]
- scoring / "U-style" exploration / tree-of-code-solutions — `score_candidate` + UCB beam tree [done]
- multiple languages — `sandbox` runner map (Python deep; bash/js/ts/ruby/go/lua structural) [done]

Wiring & control

- New executor action **`CODE_SOLVE`** (`args.description`, optional `args.language`) saves the solution to `artifacts/scripts/` and returns code + provenance. It is **additive** — `GENERATE_SCRIPT` and the self-patch engine are untouched.
- Env knobs: `ELI_CODING_SANDBOX` (default on; 0 disables execution), `ELI_CODING_RUN_TIMEOUT` (20s), `ELI_CODING_BEAM` (3), `ELI_CODING_MAX_ITERS` (6).
- Bug memory DB: `<db_dir>/coding_memory.sqlite3`.

Honest scope (what's verified vs model-dependent)

- **Verified deterministically** (tests in `tests/test_coding_agent.py`, no model): the sandbox crash/tolerate logic, the bug classifier + memory recall, the gate ladder + scoring, and the full plan→search→repair→record loop driven by a stub generator (buggy seed → UCB refinement → correct, fix recorded).
- **Model-dependent quality:** the *content* of plans, implementations, and synthesized tests scales with the local model. The machinery is frontier-shaped regardless of model.
- **Depth:** execution-level verification is Python-first; other languages run if their interpreter is installed (structural support).
- **Subtask DAG (added):** decomposable tasks are solved as a dependency graph of subtasks on the shared `eli/core/dag` engine (`plan_graph.solve_dag`), composed into one module — see `dag.md`. Gated by `ELI_CODING_DAG` (default on); cohesive tasks fall back to single-shot.
- **Chat routing (added):** `CODE_SOLVE` is now reachable from chat for explicit intents ("solve this coding problem", "implement X and test it", "use the coding agent", "verified solution"). Generic "write a python function" still routes to the lighter `GENERATE_SCRIPT` (an earlier router path claims it — precedence is tunable but the router was intentionally left unreordered).
- **Not yet:** composition is single-module (ordered concat + import-dedupe), not true multi-file projects; test synthesis verifies against synthesized tests, not a user-supplied spec suite.

FIX_FILE (`executor_enhanced.py`, not this package, but shares its safety net)

`FIX_FILE` — the directly user-triggered "fix/improve this file" action — routes Python fixes through this package's `solve()` (syntax + lint + execution + repo-context + self-critique), then applies the result to an *existing* project file. It now carries the same safety net `self_improvement.py`'s autonomous patcher has: refuses to touch the protected guardrail-file list (`is_protected_patch_path`), smoke-imports the patched module before accepting it, runs targeted tests, and rolls back to the pre-fix backup on either failure — all reusing `self_improvement.py`'s own verification functions rather than a second implementation. Worth stating plainly: since autonomous self-patching is opt-in and off by default, FIX_FILE is almost certainly the *more* exercised of the two real-file-editing paths, so it needed to be at least as protected, not less.